

Reader agency and permutation structure — computational companion

Driven by Dean A. Kulik, QuHarmonics Research Group (ORCID 0009-0003-3128-8828)

Version 3, September 2026. Reproduces every number in the paper, including the new §4.1.1 (Remark 4.5.1, Corollary 4.5.2) and the full twelve-pair range of Proposition 5.1.

Self-contained: standard library only, no files read, no downloads. Run the cells in order; the default setting takes about a minute. Set `RUN_LARGE = True` to add the $N=10$ checks (a few more minutes).

Conventions. Cells are $0, \dots, N-1$; a permutation is a tuple `p` with `p[x]` the image of `x`. A *cut* is a labelling of cells by blocks; a *balanced two-block cut* splits the cells into two halves. A reader returns the block-to-block table of counts.

Changes from version 2. Three checks are new (§4.1.1 census, §4.1.1 iterate algebra, Proposition 5.1 at every $N \leq 8$). One earlier claim is corrected in place and labelled; see the Corollary 4.5.2 cell.

```
In [1]: RUN_LARGE = False
import itertools, math, random, time
from collections import Counter, defaultdict
from math import lgamma, log, log2, lcm
from fractions import Fraction

def log2_factorial(k): return lgamma(k + 1) / log(2)

def cycles_of(p):
    n = len(p); seen = [False] * n; out = []
    for i in range(n):
        if not seen[i]:
            c = []; j = i
            while not seen[j]:
                seen[j] = True; c.append(j); j = p[j]
            out.append(c)
    return out

def cycle_type(p): return tuple(sorted((len(c) for c in cycles_of(p)), reverse=True))
def order_of(p): return lcm(*cycle_type(p))
def k_of(p): return sum(1 for c in cycles_of(p) if len(c) >= 3)
def inverse(p):
    q = [0] * len(p)
    for i, v in enumerate(p): q[v] = i
    return tuple(q)
def compose(p, q): return tuple(p[q[x]] for x in range(len(p)))
def power(p, t):
    r = tuple(range(len(p)))
```

```

    for _ in range(t): r = compose(p, r)
    return r

def table(p, m):
    '''reader with m equal contiguous blocks'''
    N = len(p); s = N // m; T = [[0] * m for _ in range(m)]
    for x, y in enumerate(p): T[x // s][y // s] += 1
    return tuple(map(tuple, T))

def balanced_cuts(N):
    '''every way to split the cells into two halves (each cut listed once)'''
    out = []; seen = set()
    for S in itertools.combinations(range(1, N), N // 2 - 1):
        S = frozenset((0,) + S)
        if S not in seen:
            seen.add(S); out.append(tuple(1 if x in S else 0 for x in range(N)))
    return out

def cut_reading(p, lab):
    '''the only free entry of a balanced two-block table: cells of block 0 sent to
    return sum(1 for x, y in enumerate(p) if lab[x] == 0 and lab[y] == 0)

def signature(p, cuts): return tuple(cut_reading(p, lab) for lab in cuts)

def reverse_cycles(p, which):
    q = list(p)
    for c in which:
        for a, b in zip(c, c[1:] + c[:1]): q[b] = a
    return tuple(q)

def reversal_orbit(p):
    cyc = [c for c in cycles_of(p) if len(c) >= 3]
    return {reverse_cycles(p, sub) for r in range(len(cyc) + 1)
            for sub in itertools.combinations(cyc, r)}

def hidden_bits(groups, total):
    return sum(len(v) * log2(len(v)) for v in groups.values()) / total

def edge_matrix(p):
    N = len(p); B = [[0]*N for _ in range(N)]
    for x, y in enumerate(p):
        if x == y: B[x][x] += 1
        else: B[min(x,y)][max(x,y)] += 1
    for u in range(N):
        for v in range(u): B[u][v] = B[v][u]
    return tuple(map(tuple, B))

print("definitions loaded")

```

definitions loaded

Proposition 2.3 — a balanced two-block reading depends only on the undirected edges

```
In [2]: rng = random.Random(0)
bad = 0
for N in (6, 8, 10):
    cuts = balanced_cuts(N)
    for _ in range(200):
        p = list(range(N)); rng.shuffle(p); p = tuple(p)
        for q in reversal_orbit(p):
            bad += signature(q, cuts) != signature(p, cuts)
print(f"reversing any subset of cycles changes a balanced two-block reading: {bad}")
```

reversing any subset of cycles changes a balanced two-block reading: 0 times out of many

Theorem 3.1 — orbit sizes of independent cycle reversal

```
In [3]: for N in (4, 6, 8):
perms = list(itertools.permutations(range(N)))
bad = sum(len(reversal_orbit(p)) != 2 ** k_of(p) for p in perms)
dist = Counter(len(reversal_orbit(p)) for p in perms)
print(f"N={N}: orbit size equals 2^(cycles of length >= 3) for all {len(perms)}
      f" | orbit-size distribution {dict(sorted(dist.items()))}")
```

N=4: orbit size equals $2^{(\text{cycles of length} \geq 3)}$ for all 24 permutations: True | orbit-size distribution {1: 10, 2: 14}
N=6: orbit size equals $2^{(\text{cycles of length} \geq 3)}$ for all 720 permutations: True | orbit-size distribution {1: 76, 2: 604, 4: 40}
N=8: orbit size equals $2^{(\text{cycles of length} \geq 3)}$ for all 40320 permutations: True | orbit-size distribution {1: 764, 2: 30008, 4: 9548}

Corollary 4.4 and Table 2 — what the family of all balanced cuts determines

```
In [4]: for N in (4, 6, 8):
perms = list(itertools.permutations(range(N))); cuts = balanced_cuts(N)
g = defaultdict(list)
for p in perms: g[signature(p, cuts)].append(p)
exact = all(set(v) == reversal_orbit(v[0]) for v in g.values())
sizes = Counter(len(v) for v in g.values())
fixes = all(len({cycle_type(x) for x in v}) == 1 for v in g.values())
print(f"N={N}: {len(cuts):>3} cuts | classes {len(g):>6} | class equals the rev
      f"cycle type determined: {fixes} | class sizes {dict(sorted(sizes.items()))}
      f"hidden {hidden_bits(g, len(perms)):.3f} bits")
```

N=4: 3 cuts | classes 11 | class equals the reversal orbit: False | cycle type determined: True | class sizes {1: 4, 2: 6, 8: 1} | hidden 1.500 bits
N=6: 10 cuts | classes 378 | class equals the reversal orbit: False | cycle type determined: True | class sizes {1: 76, 2: 282, 4: 20} | hidden 1.006 bits
N=8: 35 cuts | classes 18155 | class equals the reversal orbit: True | cycle type determined: True | class sizes {1: 764, 2: 15004, 4: 2387} | hidden 1.218 bits

```
In [5]: N = 8
b = Counter(cycle_type(p) for p in itertools.permutations(range(N)) if k_of(p) == 2
```

```
print("Table 2 (size-4 classes at N=8):", {k: v // 4 for k, v in sorted(b.items())}
      "total", sum(b.values()) // 4)
```

Table 2 (size-4 classes at N=8): {(3, 3, 1, 1): 280, (3, 3, 2): 280, (4, 3, 1): 840, (4, 4): 315, (5, 3): 672} total 2387

Table 1 — the hierarchy: fixed cut, iteration, movable cut

hidden is the conditional entropy $H(T \mid O)$ under the uniform distribution on S_N , equal to the expected $\log_2$ of the observation-class size.

```
In [6]: def hierarchy(N, horizon):
        perms = list(itertools.permutations(range(N))); cuts = balanced_cuts(N); lab0 =
        lev = {k: defaultdict(list) for k in range(1, 5)}
        for p in perms:
            pw = []; q = tuple(p)
            for _ in range(horizon):
                pw.append(q); q = compose(p, q)
            lev[1][cut_reading(p, lab0)].append(p)
            lev[2][tuple(cut_reading(r, lab0) for r in pw)].append(p)
            lev[3][signature(p, cuts)].append(p)
            lev[4][tuple(cut_reading(r, l) for r in pw for l in cuts)].append(p)
        names = {1: "one reading, fixed cut", 2: "all iterates, fixed cut",
                  3: "one reading, every cut", 4: "all iterates, every cut"}
        print(f"N={N}: {len(perms)} permutations, {len(cuts)} balanced cuts, horizon {h}
        for k in (1, 2, 3, 4):
            g = lev[k]
            fixes = all(len({cycle_type(x) for x in v}) == 1 for v in g.values())
            print(f" {names[k]:>24}: classes {len(g):>6} hidden {hidden_bits(g, len(p
            f"cycle type determined: {fixes}")
        hierarchy(6, 6)
        hierarchy(8, 15)           # 15 is the largest order of a permutation of 8 cells
```

```
N=6: 720 permutations, 10 balanced cuts, horizon 6
    one reading, fixed cut: classes      4  hidden   8.023 bits  cycle type determin
ed: False
    all iterates, fixed cut: classes     14  hidden   5.770 bits  cycle type determin
ed: False
    one reading, every cut: classes     378  hidden   1.006 bits  cycle type determin
ed: True
    all iterates, every cut: classes     378  hidden   1.006 bits  cycle type determin
ed: True
N=8: 40320 permutations, 35 balanced cuts, horizon 15
    one reading, fixed cut: classes      5  hidden  13.657 bits  cycle type determin
ed: False
    all iterates, fixed cut: classes     40  hidden  10.066 bits  cycle type determin
ed: False
    one reading, every cut: classes    18155  hidden   1.218 bits  cycle type determin
ed: True
    all iterates, every cut: classes    18155  hidden   1.218 bits  cycle type determin
ed: True
```

Proposition 4.5 — the exceptions, and where the extra collisions come from

```
In [7]: for N in (4, 6):
        perms = list(itertools.permutations(range(N))); cuts = balanced_cuts(N)
        g = defaultdict(list)
        for p in perms: g[signature(p, cuts)].append(p)
        extra = [v for v in g.values() if set(v) != reversal_orbit(v[0])]
        print(f"N={N}: {len(extra)} of {len(g)} classes are larger than a reversal orbit")
        for v in extra[:1]:
            orbs = {frozenset(reversal_orbit(x)) for x in v}
            print(f"    size {len(v):>2}, merging {len(orbs)} reversal orbits, "
                  f"    cycle types {sorted(set(map(cycle_type, v)))}")
```

N=4: 4 of 11 classes are larger than a reversal orbit
size 2, merging 2 reversal orbits, cycle types [(2, 1, 1)]
N=6: 10 of 378 classes are larger than a reversal orbit
size 4, merging 2 reversal orbits, cycle types [(3, 1, 1, 1)]

Remark 4.5.1 (NEW) — every anomaly is a complementary-support pair

The claim: with $m=N/2$, each class strictly larger than its reversal orbit consists of an m -cycle on some m -subset S together with the m -cycle on the complement, each fixing the other side pointwise. The one $N=4$ class of size 8 is explicitly *not* of this form and is reported separately.

```
In [8]: for N in (4, 6):
        m = N // 2
        perms = list(itertools.permutations(range(N))); cuts = balanced_cuts(N)
        g = defaultdict(list)
        for p in perms: g[signature(p, cuts)].append(p)
        anomalies = [v for v in g.values() if len(v) != 2 ** k_of(v[0])]
        matched, unmatched = [], []
        for v in anomalies:
            supports = {frozenset(c) for p in v for c in cycles_of(p) if len(c) == m}
            ok = (len(supports) == 2 and
                  list(supports)[0] == frozenset(range(N)) - list(supports)[1] and
                  all(len([c for c in cycles_of(p) if len(c) >= 2]) == 1 for p in v))
            (matched if ok else unmatched).append(v)
        print(f"N={N}, m={m}: {len(anomalies)} anomalous classes | "
              f"complementary-support pairs: {len(matched)} | other: {len(unmatched)}")
        for v in unmatched:
            sup = {len(c) for p in v for c in cycles_of(p) if len(c) >= 2}
            print(f"    NOT an instance of the identity: class of size {len(v)}, "
                  f"    cycle types {sorted(set(map(cycle_type, v)))}, support sizes {sorted(sup)} "
                  f"vs block size {m}")
```

N=4, m=2: 4 anomalous classes | complementary-support pairs: 3 | other: 1

NOT an instance of the identity: class of size 8, cycle types [(3, 1)], support sizes [3] vs block size 2

N=6, m=3: 10 anomalous classes | complementary-support pairs: 10 | other: 0

Corollary 4.5.2 (NEW, and a labelled correction) — the mergers survive duration

Two regimes. For $m \mid t$ both T^t and T'^t are again complementary m -cycles and D is the same nonzero matrix as at $t=1$; for $m \nmid t$ both collapse to the identity and $D=0$. Either way the signatures agree, so no horizon splits the merger.

Correction. An earlier draft asserted D was the identical nonzero matrix at every t . That is false on the multiples of m . The conclusion is unchanged; the reason differs there.

```
In [9]: for N, S, F in [(4, [0,1], [2,3]), (6, [0,1,2], [3,4,5])]:
    m = N // 2; cuts = balanced_cuts(N)
    T = list(range(N)); [T.__setitem__(S[i], S[(i+1) % m]) for i in range(m)]; T
    Tp = list(range(N)); [Tp.__setitem__(F[i], F[(i+1) % m]) for i in range(m)]; Tp
    print(f"N={N}, m={m}: T={T} T'={Tp}")
    print(f"  {'t':>3} {'m|t':>6} {'D':>9} {'signatures agree':>18}")
    Dnz, Dz, allsame = set(), set(), True
    for t in range(1, 4*m + 1):
        Tt, Tpt = power(T, t), power(Tp, t)
        B1, B2 = edge_matrix(Tt), edge_matrix(Tpt)
        D = tuple(tuple(B1[u][v] - B2[u][v] for v in range(N)) for u in range(N))
        zero = all(x == 0 for r in D for x in r)
        same = signature(Tt, cuts) == signature(Tpt, cuts)
        allsame &= same
        (Dz if t % m == 0 else Dnz).add(D)
    if t <= 2*m:
        print(f"  {t:>3} {str(t % m == 0):>6} {'0' if zero else 'nonzero':>9}")
    print(f"  distinct D over t with m does not divide t : {len(Dnz)} (1 = const)")
    print(f"  D identically zero over t with m | t : ")
    print(f"    {all(x == 0 for D in Dz for r in D for x in r)}")
    print(f"  signatures agree at every t through {4*m} : {allsame}\n")
```

```

N=4, m=2: T=(1, 0, 2, 3)  T'=(0, 1, 3, 2)
  t  m|t      D  signatures agree
  1  False  nonzero      True
  2   True    0          True
  3  False  nonzero      True
  4   True    0          True
distinct D over t with m does not divide t : 1  (1 = constant)
D identically zero over t with m | t      : True
signatures agree at every t through 8      : True

N=6, m=3: T=(1, 2, 0, 3, 4, 5)  T'=(0, 1, 2, 4, 5, 3)
  t  m|t      D  signatures agree
  1  False  nonzero      True
  2  False  nonzero      True
  3   True    0          True
  4  False  nonzero      True
  5  False  nonzero      True
  6   True    0          True
distinct D over t with m does not divide t : 1  (1 = constant)
D identically zero over t with m | t      : True
signatures agree at every t through 12     : True

```

Theorem 4.3 — the proof's two steps, checked

First: for any two permutations with the same signature, the difference of edge matrices has the form of Lemma 4.2. Second: the integrality obstruction that forces $D=0$ for even $N \geq 8$.

```

In [10]: def solve_a(B, Bp, N):
    '''test D_uv = a_u + a_v (u != v), D_uu = -(N-2)a_u/2, sum a_u = 0'''
    D = [[Fraction(B[u][v] - Bp[u][v]) for v in range(N)] for u in range(N)]
    if all(D[u][v] == 0 for u in range(N) for v in range(N)): return [Fraction(0)]*N
    a = [None]*N
    a[0] = (D[0][1] + D[0][2] - D[1][2]) / 2
    for x in range(1, N): a[x] = D[0][x] - a[0]
    ok = (all(D[x][y] == a[x] + a[y] for x in range(N) for y in range(N) if x != y)
          and all(D[x][x] == -Fraction(N-2, 2)*a[x] for x in range(N)) and sum(a) == 0)
    return a, ok

for N in (4, 6, 8):
    cuts = balanced_cuts(N); g = defaultdict(list)
    for p in itertools.permutations(range(N)): g[signature(p, cuts)].append(p)
    same_B = sum(1 for v in g.values() if len({edge_matrix(x) for x in v}) == 1)
    print(f"N={N}: classes {len(g):>6}, classes with a single edge matrix: {same_B}")
    for v in g.values():
        Bs = {edge_matrix(x): x for x in v}
        if len(Bs) > 1:
            (B1, p1), (B2, p2) = list(Bs.items())[:2]
            a, ok = solve_a([list(r) for r in B1], [list(r) for r in B2], N)
            print(f"    example {p1} vs {p2}: a = {[str(x) for x in a]}, Lemma 4.2 f
            break
    print()

```

```

print(" N    q=2/(N-2)    nonzero a with every a_u+a_v integral?")
for N in range(4, 21, 2):
    q = Fraction(2, N-2); found = None
    for npos in range(N+1):
        for nneg in range(N+1-npos):
            nzero = N - npos - nneg
            if npos*q != nneg*q or npos + nneg == 0: continue
            sums = set()
            if nzero and (npos or nneg): sums.add(q)
            if npos >= 2 or nneg >= 2: sums.add(2*q)
            if all(x.denominator == 1 for x in sums): found = (npos, nneg, nzero);
        if found: break
    print(f"{N:>3}    {str(q):>9}    {found if found else 'none -> D = 0, Theorem 4.3"}

```

N=4: classes 11, classes with a single edge matrix: 7
 example (0, 1, 3, 2) vs (1, 0, 2, 3): a = ['-1', '-1', '1', '1'], Lemma 4.2 form holds: True

N=6: classes 378, classes with a single edge matrix: 368
 example (0, 1, 2, 5, 3, 4) vs (2, 0, 1, 3, 4, 5): a = ['-1/2', '-1/2', '-1/2', '1/2', '1/2', '1/2'], Lemma 4.2 form holds: True

N=8: classes 18155, classes with a single edge matrix: 18155

N	q=2/(N-2)	nonzero a with every a_u+a_v integral?
4	1	(1, 1, 2)
6	1/2	(3, 3, 0)
8	1/3	none -> D = 0, Theorem 4.3 applies
10	1/4	none -> D = 0, Theorem 4.3 applies
12	1/5	none -> D = 0, Theorem 4.3 applies
14	1/6	none -> D = 0, Theorem 4.3 applies
16	1/7	none -> D = 0, Theorem 4.3 applies
18	1/8	none -> D = 0, Theorem 4.3 applies
20	1/9	none -> D = 0, Theorem 4.3 applies

Proposition 5.1 and Table 3 — the passive causal quotient

Scope widened in version 3. The proposition is stated for every $N \leq 8$ and every proper divisor m of N — twelve pairs. All twelve are run here; Table 3 in the paper tabulates the two largest N .

```

In [11]: print(f"{N':>2} {'m':>2} {'fibers':>7} {'causal':>7} {'hid now':>9} {'hid fvr':>9}
          f"{ 'recov':>7} {'cyc type':>9} {'#cycles':>8} {'order':>6}")
allgood = True
for N in range(2, 9):
    perms = list(itertools.permutations(range(N)))
    H = max(order_of(p) for p in perms)
    for m in [d for d in range(1, N) if N % d == 0]:
        one = defaultdict(list); fut = defaultdict(list)
        for p in perms:
            seq = []; q = tuple(p)
            for _ in range(H):
                seq.append(table(q, m)); q = compose(p, q)
            one[seq[0]].append(p); fut[tuple(seq)].append(p)
        hn, hf = hidden_bits(one, len(perms)), hidden_bits(fut, len(perms))

```

```

d_ct = all(len({cycle_type(x) for x in v}) == 1 for v in fut.values())
d_nc = all(len({len(cycles_of(x)) for x in v}) == 1 for v in fut.values())
d_ord = all(len({order_of(x) for x in v}) == 1 for v in fut.values())
allgood &= not (d_ct or d_nc or d_ord)
print(f"{N:>2} {m:>2} {len(one):>7} {len(fut):>7} {hn:>9.3f} {hf:>9.3f} "
      f"{hn-hf:>7.3f} {str(d_ct):>9} {str(d_nc):>8} {str(d_ord):>6}")

print()
print(f"Proposition 5.1 holds on all twelve (N,m) pairs: {allgood}")

```

N	m	fibers	causal	hid now	hid fvr	recov	cyc type	#cycles	order
2	1	1	1	1.000	1.000	0.000	False	False	False
3	1	1	1	2.585	2.585	0.000	False	False	False
4	1	1	1	4.585	4.585	0.000	False	False	False
4	2	3	5	3.333	2.333	1.000	False	False	False
5	1	1	1	6.907	6.907	0.000	False	False	False
6	1	1	1	9.492	9.492	0.000	False	False	False
6	2	4	14	8.023	5.770	2.253	False	False	False
6	3	21	87	5.400	3.067	2.333	False	False	False
7	1	1	1	12.299	12.299	0.000	False	False	False
8	1	1	1	15.299	15.299	0.000	False	False	False
8	2	5	40	13.657	10.066	3.591	False	False	False
8	4	282	2457	7.429	4.052	3.376	False	False	False

Proposition 5.1 holds on all twelve (N,m) pairs: True

Proposition 4.6 — how few cuts suffice

```

In [12]: def orbit_rep(p):
          cyc = [c for c in cycles_of(p) if len(c) >= 3]
          return min(reverse_cycles(p, sub) for r in range(len(cyc) + 1)
                     for sub in itertools.combinations(cyc, r))
def reading_set(p, A): return sum(1 for x, y in enumerate(p) if x in A and y in A)

for N in (4, 6, 8):
    cuts = [frozenset((0,) + S) for S in itertools.combinations(range(1, N), N // 2)]
    reps = sorted({orbit_rep(p) for p in itertools.permutations(range(N))})
    col = {A: tuple(reading_set(p, A) for p in reps) for A in cuts}
    def n_classes(fam): return len(set(zip(*[col[A] for A in fam]))) if fam else 1
    target = n_classes(cuts); best = None
    for trial in range(25):
        rng = random.Random(trial); pool = list(cuts); rng.shuffle(pool); fam = []
        for A in pool:
            fam.append(A)
            if n_classes(fam) == target: break
        i = 0
        while i < len(fam):
            t = fam[:i] + fam[i+1:]
            if t and n_classes(t) == target: fam = t
            else: i += 1
        if best is None or len(fam) < len(best): best = fam
    A0 = frozenset(range(N // 2)); canon = lambda A: A if 0 in A else frozenset(ran
    swapfam = list(dict.fromkeys([canon(A0)] +
        [canon(frozenset((A0 - {a}) | {b})) for a in A0 for b in range(N
    lb = math.ceil(math.log(target, N // 2 + 1))
    print(f"N={N}: {len(cuts)} cuts give {target} classes | counting lower bound {lb}

```

```
f"smallest family found {len(best)} | base cut plus single swaps "  
f"({len(swapfam)} cuts) gives {n_classes(swapfam)}")
```

N=4: 3 cuts give 11 classes | counting lower bound 3 | smallest family found 3 | base cut plus single swaps (3 cuts) gives 11
N=6: 10 cuts give 378 classes | counting lower bound 5 | smallest family found 9 | base cut plus single swaps (10 cuts) gives 378
N=8: 35 cuts give 18155 classes | counting lower bound 7 | smallest family found 18 | base cut plus single swaps (17 cuts) gives 14457

\$N=10\$ (set `RUN_LARGE = True`)

```
In [13]: if RUN_LARGE:  
    N = 10; cuts = balanced_cuts(N); rng = random.Random(7)  
    fam = []  
    for tri in itertools.permutations(range(N), 3):  
        if tri[0] == min(tri):  
            p = list(range(N)); p[tri[0]], p[tri[1]], p[tri[2]] = tri[1], tri[2], tri[0]  
            fam.append(tuple(p))  
    g = defaultdict(list)  
    for p in fam: g[signature(p, cuts)].append(p)  
    bad = sum(1 for v in g.values() if set(v) != (reversal_orbit(v[0]) & set(fam)))  
    print(f"N=10, all {len(fam)} permutations of cycle type (3,1^7): "  
          f"classes {len(g)}, larger than a reversal orbit: {bad}")  
    seen = {}; bad2 = 0  
    for _ in range(60000):  
        p = list(range(N)); rng.shuffle(p); p = tuple(p)  
        s = signature(p, cuts)  
        if s in seen and p not in reversal_orbit(seen[s]): bad2 += 1  
        seen.setdefault(s, p)  
    print(f"N=10, 60000 random permutations: collisions outside a reversal orbit: {bad2}")  
else:  
    print("set RUN_LARGE = True to run the N=10 checks")
```

set `RUN_LARGE = True` to run the N=10 checks